Convolutional Neural Network (CNN)

- A **Convolutional Neural Network (CNN)** is a type of deep learning model designed for processing **grid-like data** such as images.
- Instead of fully connecting every neuron to every pixel (like in Feedforward Neural Networks), CNNs use **convolutional layers** that apply small filters (kernels) across the image to detect **patterns (edges, shapes, textures)**.

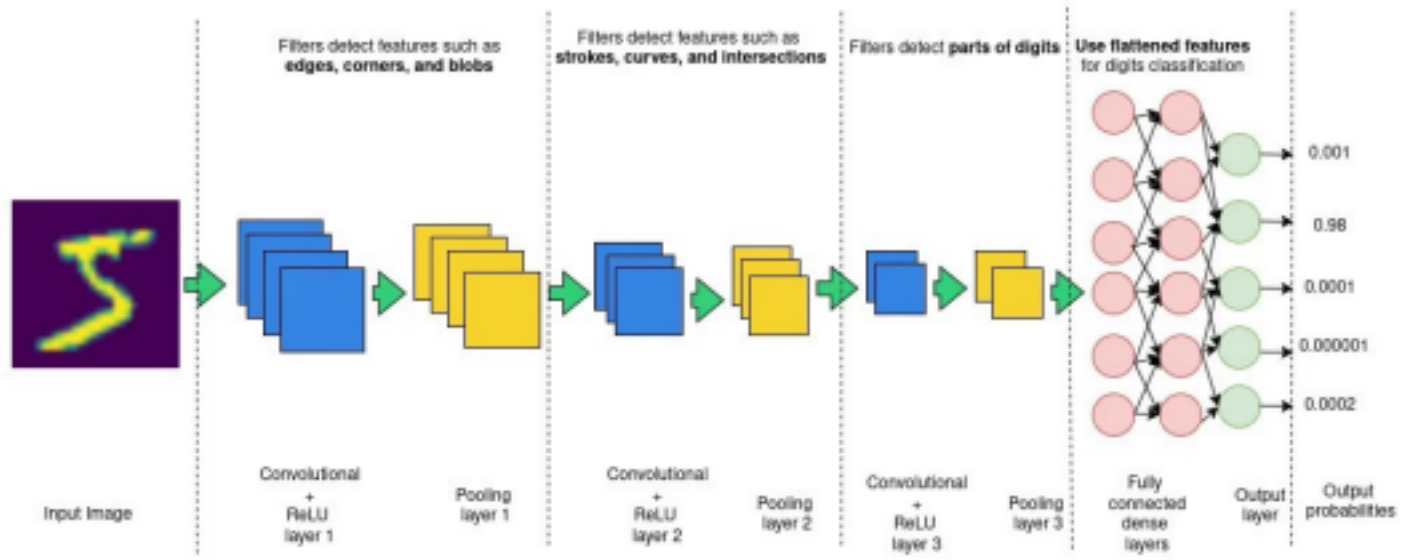
Why CNNs :

- Images are large (e.g., 224×224×3 = 150k+ values). Fully connected layers would be too big. •

CNNs exploit:

  - **Local connectivity** → look at small regions.
  - **Weight sharing** → same filter applied everywhere.
  - **Hierarchical learning** → low-level features (edges) → high-level features (objects).

Key Components of CNN



Convolution Layer

- Applies filters (small matrices like 3×3, 5×5) across the image.
- Each filter extracts specific features (edges, corners, textures).

Example filter (edge detector):

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Pooling Layer

- Reduces the size of feature maps.
- **Max Pooling** (most common): takes the maximum value in a small window (e.g., 2×2). •

Helps with **dimensionality reduction** and prevents overfitting.

Output Layer

- Usually uses **Softmax** for classification.
- Example: For 10 classes → softmax gives 10 probabilities.

Input Image

- Images are stored as **pixel grids**.
- Grayscale → 2D matrix of values (0–255).
- RGB → 3D matrix (Height × Width × 3 channels).
  Example: `224×224×3` for a color image.

Convolution

- The main operation in CNN.
- A **filter/kernel** (small matrix, e.g., 3×3) slides over the image.
- Computes a **dot product** → creates a **feature map**.
- Helps detect features like **edges, corners, textures**.

Formula:

Feature Map=∑(Input×Kernel)

Kernel / Filter

- Small matrix (e.g., 3×3, 5×5) with weights.
- Shared across the image → fewer parameters than fully connected networks. • Each kernel detects a **specific feature**.

Feature Map

- Output of a convolution operation.
- Shows **where the filter detected its pattern** in the image.

Stride

- Number of steps the filter moves when sliding.
- **Stride = 1** → moves 1 pixel at a time (large feature map).
- **Stride = 2** → moves 2 pixels at a time (smaller feature map).

Padding

- Adding extra pixels around the border of the image.
- Prevents shrinking of output after convolution.
- **Same padding** → output same size as input.
- **Valid padding** → no padding; output shrinks.

Channels

- **Depth** of the input image.
- Example: RGB = 3 channels.
- CNN learns **multiple filters per layer** (e.g., 32 filters → 32 feature maps).

Activation Function (ReLU)

- Introduces non-linearity.
- Most common: **ReLU** → replaces negative values with 0.

$f(x)=\max(0,x)$

Pooling

- Reduces size of feature maps → less computation, avoids overfitting.
- **Max Pooling**: takes max value in a region.

- **Average Pooling**: takes average.

Example (2×2 Max Pooling):

```
[1 3] 6
[5 6] → (max value)
```

Flattening

- Converts 2D feature maps into a 1D vector.
- Required before connecting to fully connected (dense) layers.

Fully Connected Layer (Dense)

- Works like a regular neural network.
- Combines extracted features → predicts final class.

Softmax Layer

- Final layer for classification tasks.
- Converts raw scores into probabilities (sums to 1).
- Example: Cat → 0.85, Dog → 0.10, Car → 0.05.

Epoch

- One full pass of training data through the network.

Batch Size

- Number of samples processed before updating model weights.

Loss Function

- Measures error between predicted and actual labels.
- Common for CNN: **Cross-Entropy Loss**.

Optimizer

- Algorithm that updates weights to minimize loss.
- Examples: **SGD, Adam, RMSprop**.

Backpropagation

- Process of calculating gradients and updating weights using **chain rule**.
- CNNs use **backpropagation through convolution and pooling layers**.

Dropout

- Regularization technique → randomly drops neurons during training.
- Prevents overfitting.

Data Augmentation

- Artificially increasing dataset by applying transformations:
    - Rotation, flipping, zooming, cropping.
- Helps CNN generalize better.

Transfer Learning

- Using a pre-trained CNN (like VGG16, ResNet, MobileNet) and fine-tuning it for a new task. • Saves training time and resources.

CNN Architectures

- Famous CNN models:
    - **LeNet-5** → First CNN (1998).
    - **AlexNet** → Won ImageNet 2012.
    - **VGGNet** → Deep, simple 3×3 filters.
    - **ResNet** → Introduced skip connections.
    - **MobileNet** → Lightweight for mobile/edge devices.

CNN Architecture (Typical Flow)

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Output